

Start Building AI Agents with MCP

A hands-on Python guide to the Model Context Protocol — from core concepts to working, runnable agent demos.

By Chandan · <https://www.linkedin.com/in/chandantechie>

Python 3.10+ · MCP Python SDK · Anthropic Python SDK

Table of Contents

1. Problems This Guide Solves
2. What Are AI Agents, and Why MCP?
3. MCP Architecture at a Glance
4. Setting Up Your Python Environment
5. Building Your First MCP Server
6. Building an MCP Client (the Agent Side)
7. Wiring an LLM Agent to Your MCP Tools
8. Three Working Demos
9. Error Handling, Security & Best Practices
10. Where to Go Next

1. Problems This Guide Solves

Before writing any code, it's worth naming exactly what goes wrong without a standard like MCP — because every piece of this guide exists to fix one of these problems.

- **Every tool needs a custom integration.** Wiring a filesystem, a database, and a weather API into one LLM app usually means three different bespoke integrations, each rewritten for the next project or the next model provider.
- **Tools aren't discoverable at runtime.** Without a shared protocol, an agent can't ask a tool provider “what can you do?” — the list of tools and their input formats has to be hand-maintained and kept in sync.
- **No separation between reasoning and execution.** Tool logic and agent logic end up tangled in one script, so a change to how a tool works risks breaking the agent loop, and vice versa.
- **Language and vendor lock-in.** A tool written for one LLM SDK typically can't be reused by a different agent framework or a different model provider without a rewrite.
- **Unsafe, ad-hoc tool calls.** Without a defined schema and boundary, it's easy to let model output reach a filesystem path, a shell command, or a SQL query with no validation in between.

MCP addresses the first four by standardizing how tools are exposed and discovered; Sections 4–7 build a real server and agent around that standard, and Section 9 addresses the last problem — safety — directly.

2. What Are AI Agents, and Why MCP?

An **AI agent** is a program that uses a large language model (LLM) not just to generate text, but to decide which actions to take, call tools, look at the results, and decide what to do next — in a loop — until a task is done.

The **Model Context Protocol (MCP)** is an open protocol (originally released by Anthropic) that standardizes how applications provide context, tools, and data to LLMs. Instead of writing a custom integration for every tool your agent needs, you expose those tools once as an **MCP server**, and any MCP-compatible client or agent can discover and call them — which is exactly the reusability and discoverability gap described in Section 1.

NOTE: This guide uses the official **MCP Python SDK** (`pip install mcp`) together with the **Anthropic Python SDK** to build a real tool-calling agent.

3. MCP Architecture at a Glance

MCP follows a simple client–host–server model:

- **MCP Server** — exposes *tools* (functions the agent can call), *resources* (data it can read), and *prompts* (reusable templates).

- **MCP Client** — embedded inside a host application; speaks the MCP protocol to one or more servers.
- **Host / Agent** — the application that owns the LLM conversation loop, decides when to call a tool, and shows results to the user.

Transport happens over **stdio** (perfect for local tools launched as a subprocess) or **Streamable HTTP / SSE** (for remote servers). Every request and response underneath is **JSON-RPC 2.0** — the SDK handles the wire format for you.



Figure 3.1 — Host, MCP client, and MCP server relationship.

4. Setting Up Your Python Environment

You need Python 3.10+ and two packages: the MCP SDK, and the Anthropic SDK for the LLM reasoning side of the agent.

```

# Create and activate a virtual environment
python3 -m venv mcp-agent-env
source mcp-agent-env/bin/activate      # Windows: mcp-agent-env\Scripts\activate

# Install dependencies
pip install "mcp[cli]" anthropic python-dotenv httpx

# Verify the MCP CLI is available
mcp --help
  
```

Terminal — one-time environment setup.

NOTE: Set your API key once via environment variable, e.g. `export ANTHROPIC_API_KEY=sk-ant-...` (or put it in a `.env` file loaded with `python-dotenv`).

Recommended project layout

```

mcp-agent-demo/
|-- server.py      # the MCP server: exposes tools
  
```

```
|-- agent.py          # the MCP client + LLM agent loop
|-- requirements.txt
`-- .env              # ANTHROPIC_API_KEY=...
```

5. Building Your First MCP Server

We'll build a small but genuinely useful server: it exposes a **weather lookup tool** and a **local file-reading tool**. Both are common building blocks for real agents, and both are simple enough to run with zero external accounts (the weather tool uses the free, keyless Open-Meteo API).

server.py — full source

```
"""
server.py - a minimal MCP server exposing two tools:
  1. get_weather(city) -> current weather via Open-Meteo (no API key needed)
  2. read_local_file(path) -> reads a text file from a sandboxed directory
"""

import os
import httpx
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("demo-tools")

# A sandboxed directory the file tool is allowed to read from.
SAFE_DIR = os.path.abspath("./sandbox")
os.makedirs(SAFE_DIR, exist_ok=True)

@mcp.tool()
async def get_weather(city: str) -> str:
    """Get the current weather for a city name.

    Args:
        city: The city name, e.g. "Bengaluru" or "London".
    """
    async with httpx.AsyncClient(timeout=10) as client:
        geo = await client.get(
            "https://geocoding-api.open-meteo.com/v1/search",
            params={"name": city, "count": 1},
        )
        geo.raise_for_status()
        results = geo.json().get("results")
        if not results:
```

```

        return f"Could not find a location named '{city}'."

    lat, lon = results[0]["latitude"], results[0]["longitude"]
    display_name = results[0]["name"]

    wx = await client.get(
        "https://api.open-meteo.com/v1/forecast",
        params={"latitude": lat, "longitude": lon, "current_weather": True},
    )
    wx.raise_for_status()
    current = wx.json()["current_weather"]

    return (
        f"Weather in {display_name}: {current['temperature']}°C, "
        f"wind {current['windspeed']} km/h."
    )

@mcp.tool()
def read_local_file(path: str) -> str:
    """Read a text file from the sandboxed 'sandbox' directory.

    Args:
        path: Filename relative to the sandbox directory, e.g. "notes.txt".
    """
    full_path = os.path.abspath(os.path.join(SAFE_DIR, path))

    if not full_path.startswith(SAFE_DIR):
        return "Error: access outside the sandbox directory is not allowed."

    if not os.path.exists(full_path):
        return f"Error: file '{path}' was not found in the sandbox directory."

    with open(full_path, "r", encoding="utf-8") as f:
        return f.read()

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

NOTE: FastMCP (part of the official SDK) turns a plain Python function into an MCP tool using its signature, type hints, and docstring — the docstring becomes the tool description the LLM sees, so write it clearly.

6. Building an MCP Client (the Agent Side)

The client's job is to launch the server, discover its tools, and forward tool calls back and forth. The MCP SDK's `ClientSession` handles the protocol; you just point it at how to start the server.

mcp_client.py — connecting and listing tools

```
"""
mcp_client.py – helper that starts server.py as a subprocess and
opens an MCP ClientSession over stdio.
"""

from contextlib import asynccontextmanager
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER_PARAMS = StdioServerParameters(
    command="python",
    args=["server.py"],
)

@asynccontextmanager
async def mcp_session():
    """Yields a ready-to-use ClientSession connected to server.py."""
    async with stdio_client(SERVER_PARAMS) as (read_stream, write_stream):
        async with ClientSession(read_stream, write_stream) as session:
            await session.initialize()
            yield session
```

7. Wiring an LLM Agent to Your MCP Tools

This is where it becomes an **agent**: we hand the tool list to Claude, let it decide when to call a tool, execute that call through the MCP session, feed the result back, and repeat until Claude gives a final answer.

agent.py — the agent loop

```
"""
agent.py – a minimal but complete tool-calling agent that uses:
* MCP (via mcp_client.py) for tool execution
* Anthropic's Claude for reasoning and deciding which tool to call

Run:
```

```

python agent.py "What's the weather like in Tokyo right now?"
"""

import sys
import asyncio
from dotenv import load_dotenv
load_dotenv()

from anthropic import Anthropic
from mcp_client import mcp_session

client = Anthropic() # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-sonnet-4-5"

def mcp_tools_to_anthropic_schema(mcp_tools):
    """Convert MCP tool definitions into the JSON schema Claude expects."""
    return [
        {
            "name": t.name,
            "description": t.description or "",
            "input_schema": t.inputSchema,
        }
        for t in mcp_tools
    ]

async def run_agent(user_message: str, max_turns: int = 6) -> str:
    async with mcp_session() as session:
        mcp_tools = (await session.list_tools()).tools
        claude_tools = mcp_tools_to_anthropic_schema(mcp_tools)

        messages = [{"role": "user", "content": user_message}]

        for _ in range(max_turns):
            response = client.messages.create(
                model=MODEL, max_tokens=1024, tools=claude_tools, messages=messages,
            )

            tool_uses = [b for b in response.content if b.type == "tool_use"]
            if not tool_uses:
                text_blocks = [b.text for b in response.content if b.type == "text"]
                return "\n".join(text_blocks)

            messages.append({"role": "assistant", "content": response.content})

```

```

        tool_results = []
        for call in tool_uses:
            result = await session.call_tool(call.name, call.input)
            result_text = "".join(
                part.text for part in result.content if hasattr(part, "text")
            )
            tool_results.append({
                "type": "tool_result",
                "tool_use_id": call.id,
                "content": result_text,
            })

        messages.append({"role": "user", "content": tool_results})

    return (
        "Reached max_turns without a final answer – "
        "try increasing max_turns."
    )

if __name__ == "__main__":
    question = " ".join(sys.argv[1:]) or "What is the weather in Bengaluru?"
    answer = asyncio.run(run_agent(question))
    print("\nAgent answer:\n" + answer)

```

NOTE: The loop is deliberately simple: *ask model* → *check for tool_use* → *run tool via MCP* → *feed result back* → *repeat*. Every production agent framework (LangGraph, AutoGen, OpenAI's Agents SDK, etc.) is a more featureful version of exactly this loop.

8. Three Working Demos

All three demos below reuse the exact `server.py`, `mcp_client.py`, and `agent.py` from Sections 5–7 — nothing new to write. Set up once, then just change the question you ask the agent.

One-time setup

```

# requirements.txt
mcp[cli]>=1.2.0
anthropic>=0.40.0
httpx>=0.27.0
python-dotenv>=1.0.0

# .env

```

```
ANTHROPIC_API_KEY=sk-ant-api03-your-key-here
```

```
# sandbox/notes.txt (sample data used by Demo 2 and Demo 3)
Team standup notes – Sept 2026
- MCP server refactor: done
- Weather tool: added retry logic
- Next: add a calendar tool
```

Demo 1 — Single-tool call (weather only)

The simplest possible run: one question, one tool call.

```
$ python agent.py "What's the weather in Tokyo right now?"
```

Agent answer:

The current weather in Tokyo is 21.4°C with wind speeds of 11.2 km/h.

Demo 1 — Claude calls `get_weather()` once and answers directly.

Demo 2 — Single-tool call (local file only)

Same agent, same tools available — this time the question only needs the file tool, and Claude correctly leaves the weather tool unused.

```
$ python agent.py "Read notes.txt from my sandbox and summarize it in one line"
```

Agent answer:

Your notes say the MCP server refactor and weather tool retry logic are both done, and the next task is adding a calendar tool.

Demo 2 — Claude calls `read_local_file()` once and summarizes the result.

Demo 3 — Multi-tool orchestration (weather + file, combined)

The same agent again, but with a question that needs both tools in one turn — this is the case that shows real orchestration, not just single tool lookup.

```
$ python agent.py "What's the weather in Tokyo, and read notes.txt for me"
```

Agent answer:

The current weather in Tokyo is 21.4°C with wind speeds of 11.2 km/h.

Here are your notes:

Team standup notes – Sept 2026

- MCP server refactor: done
- Weather tool: added retry logic

```
- Next: add a calendar tool
```

Demo 3 — Claude decides on its own to call both `get_weather()` and `read_local_file()`.

Notice what happened across all three: you never told the agent *which* tool to use. It read the tool descriptions (pulled straight from your Python docstrings), matched them to each question, and called zero, one, or two tools as needed. That decision-making is the whole point of an agent.

9. Error Handling, Security & Best Practices

Sandbox everything the tool touches

The `read_local_file` example resolves the absolute path and rejects anything outside `SAFE_DIR`. Any tool that touches the filesystem, a shell, or a database needs an equivalent boundary — never trust a path or query string that ultimately came from model output.

Validate and bound tool inputs

- Use precise type hints; FastMCP turns them into a JSON schema that constrains what the model can send.
- Set explicit limits (max rows, max file size, request timeouts) inside every tool.
- Return clear error strings instead of raising, so the agent can see the failure and retry or explain it — don't let one bad tool call crash the whole loop.

Cap the agent loop

Always bound `max_turns` (as in Section 7) so a confused model can't loop forever burning API calls. Log every tool call and result during development — it's the fastest way to see why an agent made a particular decision.

Keep secrets out of tool descriptions and results

WATCH OUT: Tool descriptions and results are sent straight to the LLM. Never put API keys, internal hostnames, or credentials in a docstring or a returned string.

Prefer official transports

Use **stdio** for local, same-machine tools (as in this guide) and **Streamable HTTP** for remote MCP servers behind auth. Avoid inventing a custom transport — the whole value of MCP is that clients and servers built by different people already speak the same protocol.

10. Where to Go Next

- Add more tools to `server.py`: a calendar tool, a database query tool, a web-search tool.
- Swap the `stdio` transport for `Streamable HTTP` once your server needs to run remotely or be shared by multiple agents.
- Explore multi-server agents — a single client can hold sessions to several MCP servers at once and merge their tool lists.
- Look at the official MCP specification and SDK repositories for resources, prompts, and sampling — this guide covered tools, which is the most common starting point.
- Try an existing agent framework (LangGraph, the Anthropic Agent SDK, etc.) once you're comfortable with the raw loop — they wrap exactly the pattern built in Section 7.

All code in this guide is self-contained and was written to run as-is with Python 3.10+, the official `mcp` package, and the `anthropic` Python SDK. Adjust the model name and API key handling to match your account.